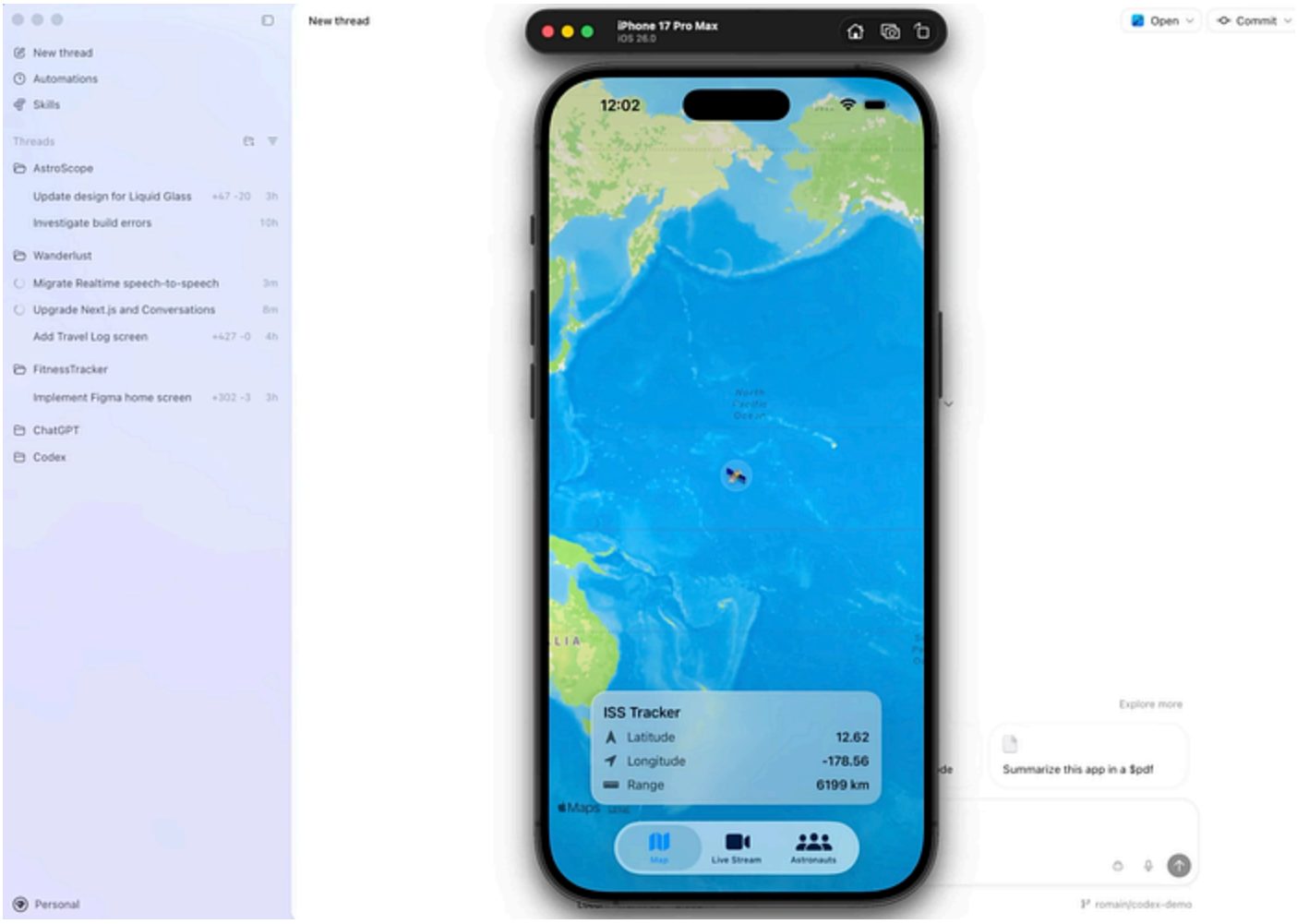
Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

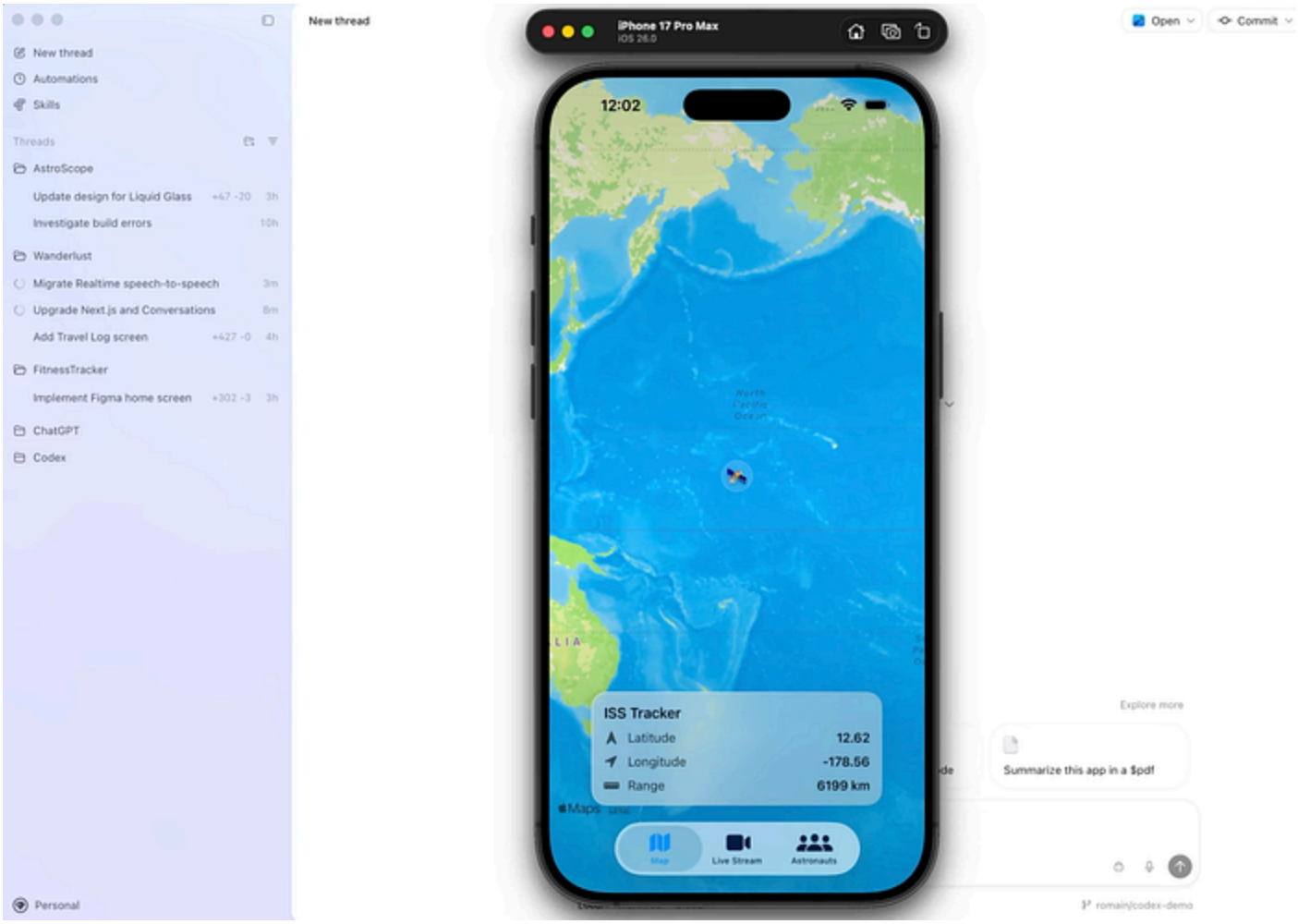
Agent Native Follow
~6 min read · February 3, 2026 (Updated: February 3, 2026) · Free: No

Developer workflows ultimately come down to **coordination cost and preventing it from compounding:**

- keeping multiple workstreams moving without stepping on each other
- controlling blast radius in a real repo
- reviewing diffs sanely
- maintaining trust boundaries (files, network, credentials)
- not losing half a day to context switches while an agent "thinks"

In this context, Codex provides better features than Claude Code because it removes friction exactly where I usually lose time.

Let me please explain.



If you haven't, please also get the first issue of [Agentic SaaS Patterns Winning in 2026](#).

The shift to "supervise a system"

The Codex app makes multi-agent orchestration feel native.

The key improvement is the **workflow architecture**: parallel threads, project switching, isolated execution contexts, resumable history across CLI/app, and explicit permission controls.

Once you experience it on a messy repository, it becomes hard to go back to one agent in one terminal tab.

Besides its pretty UI, it changes the cost structure of waiting, tracking, and supervising.

Why it's faster even when the model is slow

Models typically scan broadly, cross-reference code paths, and generate a cohesive plan.

That depth is often valuable, but it also means you're **waiting**.

The Codex app turns **waiting** into **parallel progress** by making it trivial to:

- kick off a task in one thread,
- jump to another thread or another project,
- run verification in-context,
- and come back when the long-running task finishes.

That sounds obvious until you compare it to switching multiple terminals, branches, and half-remembered which tab is running what.

So I don't think this workflow wins by making the agent faster, but it's making me **waste less time while the agent is slow**.

Developer pain around worktrees

I've said for a while that **worktrees are the battleground** for agentic development because real work is rarely linear.

You're doing a refactor while debugging a flaky test while addressing a review comment while prepping a release.

And I also admit that **Git worktrees are not a magical solution**.

They can become a mess:

- you can't have the same branch/commit checked out in two places
- syncing between worktrees can be annoying
- it gets harder to remember where "the real state" of code lives

What's interesting is that Codex doesn't treat worktrees as a sacred Git primitive, it behaves more like **task-isolated checkouts/clones** that you can later sync back.

That can be behavior-changing in the best way:

- You can delegate more confidently (because the agent isn't poking your main working copy).
- You can parallelize without fear of collisions.

- You spend less mental energy on "what did it just touch?"

But it's not perfect and this is where I'm intentionally not hand-wavy because you will also notice.

The path from **agent did work in an isolated context** to **clean PR against the branch I actually care about** isn't always as direct as you'd want.

So the ergonomics of syncing, branching-from-branch, and PR creation could be smoother in some flows.

The killer feature is app + CLI continuity

This is one of those things you only appreciate once you start using it, Codex feels like a coherent "front-end" to your agent workflow.

There are two specific aspects that matter:

- **Shared history** between CLI runs and the app
- **Shared configuration**, so the app doesn't feel like a separate universe

That continuity reduces friction in a very practical way:

- You can scope work in the CLI (fast, interactive, easy to iterate)
- then push the bigger execution into the app (parallel threads, safer isolation, better supervision)

It becomes one loop rather than two tools.

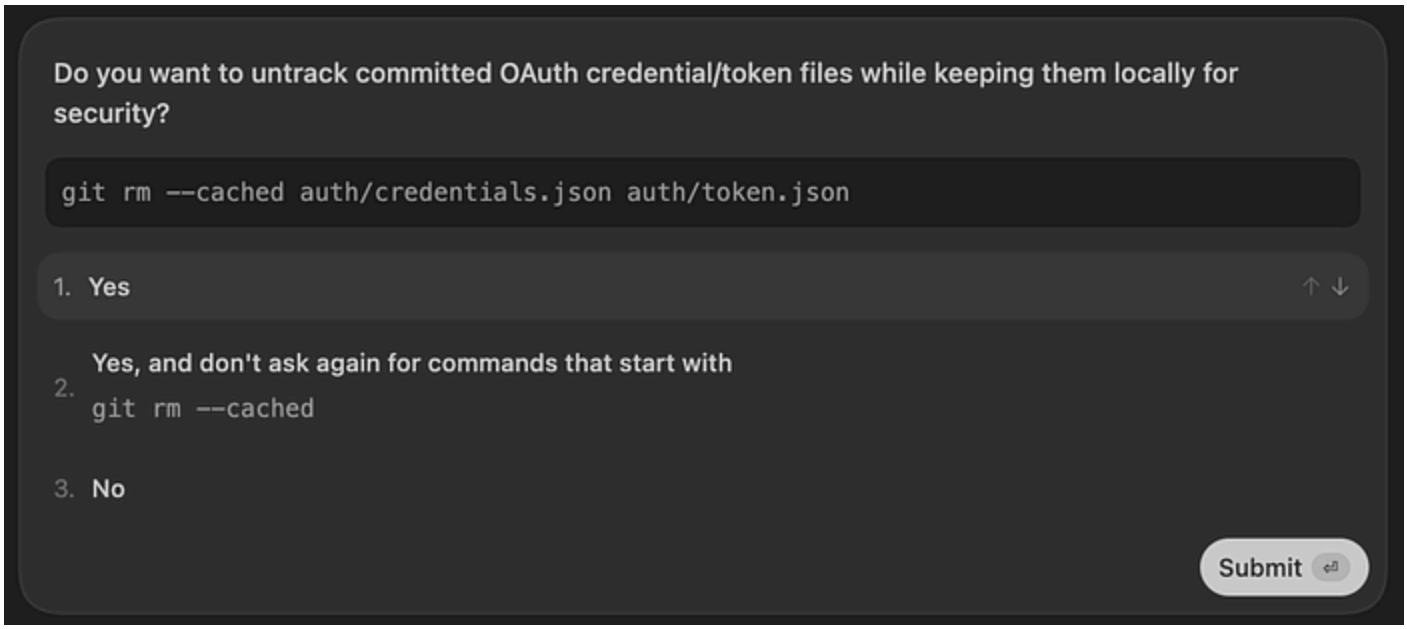
The practical stack I now use

Here's the workflow that has stuck for me:

1) Interactive scoping in CLI

I start where it's easiest to shape intent:

- clarify goal and constraints
- ask for a plan
- approve or redirect early



2) Branch-safe execution in app threads / isolated contexts

When the task is non-trivial (refactors, cross-cutting changes, multi-file edits), I push it into the app so it can run without threatening my main checkout.

3) Verification in a per-thread terminal

The per-thread terminal is underrated. It keeps verification close to the work:

- run tests and lint
- inspect build output
- do quick Git sanity checks

4) Review loop: diffs > editor > commit/PR

It is to open the changed code in the editor, review, and then drive it toward a commit/PR from the app.

5) Automations as "promptable cron jobs"

This is the other shift that feels genuinely new.

Automations are basically scheduled agent runs that operate with project context that iare useful for:

- summarizing CI failures or recent commit risk
- scanning for regressions after merges
- generating changelog drafts
- recurring triage/reporting that you'd otherwise procrastinate

They also appear as threads, with their own working context, which makes them auditable and reviewable rather than invisible background noise.

What I actually mean by "Better than Claude Code"

I'm not interested in writing a fan post that collapses under scrutiny.

There are areas where Claude Code still feels better to me.

1) "Computer tasks" that aren't repo-bound

If I'm doing one-off scripts, dotfile edits, local machine housekeeping, or ad-hoc exploration that doesn't naturally live in a Git repo, Claude Code still feels more flexible.

2) UI passes and fast visual iteration

If the task is heavy on UI tweaking, especially where I want quick local iteration, Claude Code (and other IDE-native flows) can still be the path of least resistance.

3) Environment management is still a soft spot

There is a friction around cloud environment setup: configuration, environment variables, and the general "container context" problem.

I agree this is still an unsolved part of the agent stack across the industry.

Codex has a strong orchestration layer, but the industry still needs a better standard for "agent-friendly dev environments."

So when I say "better than Claude Code," what I actually mean:

Codex currently wins on *system coherence*:

- app + CLI continuity (shared history, shared config)
- built-in parallelism that doesn't feel fragile
- isolation-first execution that reduces blast radius
- review-oriented workflow (diffs, editor, verification, PR)
- explicit permission boundaries that make trust scalable
- automations that reuse real project context

In other words, it's **less costly to operate**.

Final take

I genuinely loved the product and its direction.

Codex respects how software teams actually work with concurrent tasks, branch isolation, review loops, and controlled permissions.

That's why, today, it feels best-in-class for me.

If you disagree, I'd love to compare workflows, especially concrete cases where Claude Code still wins for you.

Let me know where do you think the real moat is for coding agents in 2026.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

"This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human

"This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human Your customer forwards an invoice PDF with the...

medium.com

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power People are building Polymarket bots, multi-agent product teams...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

The P99 Problem: Designing LLM Inference for Real Users

The P99 Problem: Designing LLM Inference for Real Users One prompt. One model. Clean hardware. No noisy neighbors. No...

medium.com
#claude-code#openai-codex#agentic-coding#ai-coding#coding

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel